



Towards Finding Accounting Errors in Smart Contracts

Brian Zhang

Purdue University

West Lafayette, USA

bzhangprogramming@gmail.com

ABSTRACT

Bugs in smart contracts may have devastating effects as they tend to cause financial loss. According to a recent study, accounting bugs are the most common kind of bugs in smart contracts that are beyond automated tools during pre-deployment auditing. The reason lies in that these bugs are usually in the core business logic and hence contract-specific. They are analogous to functional bugs in traditional software, which are largely beyond automated bug finding tools whose effectiveness hinges on uniform and machine checkable characteristics of bugs. It was also reported that accounting bugs are the second-most difficult to find through manual auditing, due to the need of understanding underlying business models. We observe that a large part of business logic in smart contracts can be modeled by a few primitive operations like those in a bank, such as deposit, withdraw, loan, and pay-off, or by their combinations. The properties of these operations can be clearly defined and checked by an abstract type system that models high-order information such as token units, scaling factors, and financial types. We hence develop a novel type propagation and checking system with the aim of identifying accounting bugs. Our evaluation on a large set of 57 existing accounting bugs in 29 real-world projects shows that 58% of the accounting bugs are type errors. Our system catches 87.9% of these type errors. In addition, applying our technique to auditing a large project in a very recent auditing contest has yielded the identification of 6 zero-day accounting bugs with 4 leading to direct fund loss.

CCS CONCEPTS

• Security and privacy → Domain-specific security and privacy architectures.

KEYWORDS

Blockchain, Smart Contract, Accounting Error, Type Checking

ACM Reference Format:

Brian Zhang. 2024. Towards Finding Accounting Errors in Smart Contracts. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, April 14–20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3597503.3639128>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE '24, April 14–20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0217-4/24/04...\$15.00
<https://doi.org/10.1145/3597503.3639128>

1 INTRODUCTION

Blockchains and cryptocurrencies have become an integral part of our economy. As of the writing of this paper, the global market cap for cryptocurrencies reaches \$1.19 Trillion USD, with the top two blockchains being Bitcoin and Ethereum [6]. An important kind of blockchain-based applications are smart contracts, which can encompass a wide range of services, from banks to gaming platforms and marketplaces. Smart contracts follow the DeFi, or *decentralized finance* principle. Unlike centralized systems, such as federal banks, smart contracts operate in a decentralized manner without a single controlling authority, rendering many novel financial applications. Similar to traditional software, smart contracts are developed by programmers and inevitably have bugs. The lucrative value of exploiting these bugs has made smart contracts one of the most popular targets of many malicious actors. As of Q2 of 2023, \$300 million USD were exploited from 212 security incidents [10], suggesting that each exploit costed an average of \$1.5 million USD.

Therefore, there is a pressing need to develop techniques to find smart contract bugs. Existing techniques can be roughly classified into four categories: static analysis, fuzzers, symbolic execution, and verification. Static analysis tools [18, 23, 26, 27, 35, 39, 60, 65, 68, 71, 79] analyze source code without actually running the code. They usually transform smart contracts to various intermediate representations and then search for certain bug patterns. Fuzzers [24, 28, 29, 34, 36, 45, 54, 67, 69, 70, 74, 75] run contracts against a large number of inputs and transactions sequences. Symbolic execution [1, 17, 19, 43, 51, 52, 55, 61, 67, 73] analyzes all possible program paths of a smart contract by performing symbolic computation instead of concrete execution. Verification tools [30, 32, 62, 63, 72] leverage formal methods to check smart contracts against formal specifications. These approaches have demonstrated great effectiveness in identifying a broad range of issues. Some bugs such as *reentrancy* and integer *overflow* and *underflow* can hardly survive these tools. However, most automatic techniques rely on application agnostic oracles, meaning that bugs need to be clearly defined without considering application specific semantics. Such oracles may be difficult to acquire for certain kinds of bugs. Verification tools are capable of detecting a wide spectrum of bugs including those that are application specific. However, they need the developers to provide application specifications, which may entail substantial manual efforts. As a result, there are still a large number of bugs that are beyond existing tools, evidenced by the growing number of exploits.

According to a recent study by Zhang et al. [78] on over 500 exploitable bugs (bugs that can lead to direct fund loss) from 119 real-world smart contract projects, 80% of exploitable bugs are *machine un-auditable bugs* (MUBs), meaning that they fall outside of the scope of existing automatic tools. Among them, *accounting bugs*, which are incorrect implementations of underlying contract

business models, are the most common type of MUBs in projects before deployment and also the second hardest to find in manual auditing, due to the need of understanding the most complex parts of contracts, namely, the business logics. On the other hand, their impact can be devastating. An example would be the Uranium Finance Exploit [14]. Due to two extra zeros in an interest calculation, the contract was exploited for \$57 million USD. The bug survived multiple rounds of manual auditing (by experts).

In this paper, we develop a type-checking tool for accounting bugs in smart contracts. Our insight is that *although accounting bugs reside in complex business logic and seemingly lack an application-agnostic oracle, many manifest themselves as abstract type violations*. *Abstract type inference* is a technique that can be traced back to the 70's in the last century [40]. It aims to abstract higher level semantic information such as physical units (e.g., seconds and meters) than those denoted by primitive types in programming languages such as integers and strings. As such, type systems can be enhanced to check a much richer set of properties [16, 31, 33, 40–42, 50, 56, 59, 61], such as physical unit consistency in robotic systems. We further observe that *although smart contracts have sophisticated business models, their basic operations are still analogous to those in a simple bank system*, such as deposit, withdraw, exchange, and loan. We hence devise an abstract type system based on these operations that can infer and check abstract types. In particular, we model and infer three facets of each variable, which are: *token unit* indicating the kind of currency denoted by the variable (analogous to USD in real life), *scaling factor* that denotes how much the variable has been scaled in order to simulate floating point computation that is not supported in smart contract programming languages, and *financial meaning*, e.g., if the variable denotes an interest or a debt. With the rich types, we can check a large set of properties that shall be uniformly true for various business models, using type rules. For instance, values of different token units cannot be added or subtracted together, similar to how lengths of meters and inches cannot be added together; amounts scaled by different factors should not be compared; interest should not be subtracted from debt but rather adds to it. More details can be found in section 3. To use our tool, the user annotates a few global variables and function parameters whose abstract types cannot be inferred from code (likely because they are implicit in pre-conditions and not reflected in implementation). The annotations are limited (see section 4) and usually clear from project description and even variable names. Then, our technique automatically infers the abstract types for other variables and performs type checking. Our system is flow-sensitive, context-sensitive and field-sensitive.

Our contributions are summarized as follows.

- We devise a novel abstract type system for smart contracts based on two observations: (1) many accounting bugs manifest themselves as abstract type errors; and (2) operations in smart contracts with complex business models can be abstracted to a few primitives and their combinations.
- Our type system models three aspects, token unit, scaling factor, and financial meaning, providing a good coverage for common type errors (according to our experiments).
- We implement a prototype `ScType` based on Slither [26]. We evaluate the system on 29 contracts from [78] that have 57 reported accounting bugs. Among these bugs, we find that

```

1  contract Pools{
2    ...
3  function addLiquidity(address base, address token, address
      member) external returns(uint liquidity) {
4      uint addedBase = getAddedAmount(base, ...);
5      uint addedToken = getAddedAmount(token, ...);
6      liquidity = calcLiquidityUnits( addedBase, totalBase,
      addedToken, totalToken, totalLiquidity);
7      liquidity[...][member] += liquidity;
8      totalBase += addedBase;    ...
9  }
10 function calcLiquidityUnits(uint b, uint B, uint t, uint T,
      uint P) external view returns (uint){
11     uint part1 = (t * B);
12     uint part2 = (T * b);
13     uint part3 = (T * B) * 2;
14     uint _units = (((P * part1) + part2) / part3);
15     return (_units) / one;
16 }

```

Figure 1: Buggy Code from *Vader* [15]

33 of them are type errors and our method detects 29 of the type errors (i.e., 87.9% recall). Our tool reports 14 false positives due to its lack of path-sensitivity. The remaining 57-33=24 bugs are mostly due to pure math incorrectness and beyond type systems. We also apply our technique in a most recent audit contest organized by Code4Rena [2] for a very large project with more than ten thousand lines of code and find six zero-day accounting bugs in nine contracts that we check. Four can lead to direct fund loss. We have included our tool and benchmarks in the supplementary document. We will release the tool upon publication.

2 MOTIVATION

We use two real-world accounting bugs to explain the inadequacy of existing techniques and illustrate our method.

Example I (Lending Contract *Vader*). Figure 1 contains code detailing two functions included within the *Pools* contract from the *Vader* project [15]. They have been shortened for demonstrative purposes. *Vader* is a lending project that enables the pooling of funds and offers borrowing functionalities. Users can participate by purchasing shares from a liquidity pool and utilize these shares as collateral to access borrowed funds, thereby increasing the pool's returns. Within the project, The Pools contract stores functions related the movement of liquidity in the pool.

The function `addLiquidity()` in Figure 1 converts a contract's recently added *base* currency and *token* currency into liquidity, and adds the liquidity to a certain member's account. In the contract, the base currency is the default currency of the *Vader* project, while the token currency is a special currency used by the liquidity pool, representing a share of the pool. The amount of recently added base currency is computed on line 4 as `addedBase`, and the amount of recently added token is computed on line 5 as `addedToken`. The equivalent amount of liquidity is calculated through the `calcLiquidityUnits()` function call on line 6, in which `totalBase`, `totalToken`, and `totalLiquidity` denote the total amount of base currency, total token currency, and total liquidity in the pool. The resulting liquidity is then added to the member account on line 7 and the total base currency is updated on line 8.

```

1  contract LibBalances{
2  ...
3  function applyTrade(Position position, Trade trade, uint256
    feeRate) internal pure returns ... {
4      int256 signedAmount = trade.amount;
5      int256 signedPrice = trade.price;
6      int256 quoteChange = signedAmount * signedPrice;
7      int256 fee = getFee(trade.amount, trade.price, feeRate);
8
9      int256 newQuote = 0;
10     int256 newBase = 0;
11     if (trade.side == LONG) {
12         newBase = position.base + signedAmount;
13         newQuote = position.quote - quoteChange + fee;
14     } else if (trade.side == SHORT) {
15         newBase = position.base - signedAmount;
16         newQuote = position.quote + quoteChange - fee;
17     }
18     ...
19 }

```

Figure 2: Buggy Code from *Tracer* [13]

The bug occurs within the function `calcLiquidityUnits()`. Associating the variables on line 6 to the formal arguments on line 10, we have that b is the recently added base currency, B is the total base currency, t is the recently added token currency, T is the total token currency, and P is the current liquidity. The function attempts to convert the base and token currencies to liquidity on lines 11-15 through the following equation: $(P * (t * B) + (T * b)) / (T * B * 2)$, which is incorrect, with the correct equation being: $P * ((t * B) + (T * b)) / (T * B * 2)$. The understanding of the exact math is unnecessary, and therefore more details are excluded. As a consequence of this bug, all conversions of currencies to liquidity are incorrect, losing the funds of both the contract and the users. This bug was ranked as *High Risk* on Code4rena, the highest severity on the platform.

Example II (Trading Contract Tracer). Figure 2 depicts a function included within the *LibBalances* library from *Tracer* [13], which is a *derivative* smart contract designed to enable users to trade in *perpetual markets* [9]. A derivative contract has its functionalities based upon derivatives, namely, financial agreements derived from an underlying asset or financial market. Examples of derivatives include stocks, options, and futures. In perpetual markets, users can place long and short trade orders aiming on buying or selling *base* tokens using *quote* tokens, respectively. Users going long earn money when the price of the base token increases, while users going short earn money when the price of the base token decreases. Every user possesses a *position* that stores the amount of base and quote tokens that the user has. The *LibBalances* contract is a library within the *Tracer* project that provides basic functionalities in its perpetual market. In particular, the function `applyTrade()` is used to finalize both long and short trades. The amount of base token associated with the trade is defined as `signedAmount` on line 4, and the price of the base token is defined in the following line 5 as `signedPrice`. The amount and price of the base token are used to calculate the equivalent amount of quote token on line 6 as `quoteChange`. Then a fee for the transaction is calculated through the `getFee()` function call on line 7.

The short trade handling from line 14 to 16 is correct, and depicts the user selling the `signedAmount` in base token and earning the `quoteChange` in quote token, minus the fee by the contract. On the other hand, the long trade handling on lines 11 to 13 is incorrect,

specifically on line 13. Instead of losing the fee meant for the contract, the long trade user gains the fee instead. A likely explanation for this bug is that the developer confused the contract fee, which is charged to the user for each transaction, with the *funding fee* [8] in perpetual contracts, which is a fee paid from long traders to short traders periodically when the base token price increases, and from short traders to long traders when the base token price decreases, as an incentive for traders. The consequence of this bug is that users entering long trades would gain additional quote tokens on the contract's loss. This bug was also ranked as *High Risk* on Code4rena.

Existing Techniques Are Insufficient. As discussed in section 1, existing techniques such as static analysis, fuzzing, and symbolic execution require application agnostic oracles, like those used in finding reentrancy bugs. Reentrancy occurs when a victim contract makes a call to an external function before updating state variables, potentially allowing the external function to call the victim contract again. This leads to a loop which can cause unexpected behaviors such as fund draining. It can be detected by finding call cycles using various analysis: static, dynamic, and symbolic. Such an oracle does not have to model application specific behaviors. However, the previous two bugs reside in the core business logic specific to the contracts. For example, the first bug is coupled with the obscure math in function `calcLiquidityUnits()` and the second bug seems to require understanding perpetual trading and the unique design of position of the *Tracer* project. They hence cannot be detected by the automated tools we have tried (see section 4).

Our Technique. To address the aforementioned limitations, We propose a novel static analysis tool for detecting accounting bugs by considering inherent financial meanings of each variable. We introduce the concept of *extended type*, which denotes information such as token unit, scaling factor, and financial type for each variable and enables type propagation. This extended type information is used to check the correctness and consistency of all operations within the contract.

We demonstrate the usage of our tool here, starting with the bug in Figure 1 regarding the function `addLiquidity()`. Through type inference and propagation, `ScType` determines the token/currency units of `addedBase` on line 4, `addedToken` on line 5, and `totalLiquidity` on line 5, as t_{base} (meaning the token denoted by the address base), t_{token} , and $e(t_{base}, t_{token})$ denoting a composite expression involving t_{base} and t_{token} . More details of such inference can be found in subsection 3.4. In addition, due to line 8, `totalBase` has a token unit of t_{base} . Similarly, `totalToken` has a unit of t_{token} . On line 6 where the function `calcLiquidityUnits()` is invoked, our tool can infer that the parameters have token units of: $\{t_{base}, t_{base}, t_{token}, t_{token}, t_e = e(t_{base}, t_{token})\}$. This information is then propagated to the `calcLiquidityUnits()` function body starting on line 10. Hence on line 11, `part1` = $t * B$, and therefore has unit $[t_{base} * t_{token}]$. On line 12, `part2` = $T * b$, and has unit $[t_{base} * t_{token}]$. On line 13, `part3` = $T * B * 2$, and therefore has unit $[t_{base} * t_{token}]$ as well. However, the issue arises on line 14 when the calculation of `_units` = $((P * \text{part1}) + \text{part2}) / \text{part3}$ is performed. This calculation is split into three separate steps: the first calculation being `TMP_0` = $P * \text{part1}$, the second calculation being `TMP_1` = `TMP_0` + `part2`, and finally, `_units` = `TMP_1` / `part3`.

Table 1: Popular DeFi Project Categories

Project Types	Brief Summary	Typical Operations
Yield & Yield Aggregator	Yield projects allow users to deposit funds to be used as collateral in other projects, where the earnings are returned.	stake, withdraw, draw, invest, reinvest
Dexes	Exchange projects allow users to trade one currency for another, or for shares in a pool.	swap, add/remove liquidity
Lending	Lending projects allow users to borrow currency in exchange for a collateral.	lend, repossess, repay loan
Services	Service projects facilitate services or functions such as games and wallets.	deposit, withdraw, buy, sell
Derivatives	Derivative projects are based upon simulating real-world derivatives such as stocks, options, and futures.	trade, liquidate, withdraw, deposit

According to the first calculation, TMP_0 has unit $[t_{base} * t_{token} * t_e]$, so during the second calculation when TMP_0 is added to part2, which has unit $[t_{base} * t_{token}]$, there is a type mismatch, and thus our tool reports an error. In contrast, the fixed version $P*((t*B) + (T*b))/(T*B*2)$ can be type-checked.

For the bug in Figure 2, the parameters of `applyTrade()` have the following initial extended type information (from its documentation): `trade.amount` is a *Balance*, denoting the amount some account owns, `trade.price` is a *Price*, denoting trading price between two products, `feeRate` is a *Fee*, denoting charge to an account by the contract, and `position.quote` is a *Balance*. The types of other variables are then inferred and checked. On line 4, the financial meaning of `signedAmount` is determined as *Balance* due to the assignment. On line 6, `quoteChange` is typed as *Balance* similarly. On line 7, `fee` is typed to *Fee* by inter-procedural analysis. Then on line 13, `position.quote - quoteChange` is typed to *Balance* as it is the difference of two balances. However, the whole expression `position.quote - quoteChange + fee` cannot be typed as *Balance + Fee* is illegal. This is because `fee` is a charge and should never be added to a balance. In contrast, line 16 can be typed.

3 DESIGN

3.1 A Conceptual DeFi Model

We studied a large number of DeFi projects (i.e. a total of 113) collected in [78]. The majority of such projects have already been deployed in the real-world, and there are many of which are fairly complicated, e.g., Tigris [12] and Biconomy [5]. We have a key observation: *many of these projects can be considered as mutations of a bank*. Banks are the oldest financial institution, and their key functionality is to collect liquidity from deposits, distribute the liquidity through loans, collect interest (and possibly collateral as well through liquidation) from those loans, and remit the profits back to the deposits. Modern DeFi projects often have fairly sophisticated business models (e.g., derivative contracts). However, their essence is still *collecting unused liquidity and re-distributing for profits*. We hence propose a bank-like conceptual model to describe some fundamental operations of DeFi projects, allowing us to derive a set of key properties. In the following, we first describe the conceptual model and then explain how existing DeFi projects can be considered as instantiations of the model. To avoid additional denotation overhead, the model is composed using a Solidity-like language

```

1  contract DeFiModel {
2      IERC20 public T; //contract token
3      IERC20 public T0; //asset token
4      uint public fee_rate;
5      uint public interest_rate;
6      uint public earning;
7      uint public totalSupply;
8      unit public totalDebt;
9      unit public price;
10     mapping(address => uint) public balance;
11     mapping(address => uint) public debt;
12     mapping(address => uint) public collateral;
13
14     function deposit(uint amount) public { //deposit T0 for T
15         unit share= swap_T0_4_T(amount);
16         uint fee= update_fee(fee_ratio, totalSupply, totalDebt);
17         balance[msg.sender]+=share;
18         balance[msg.sender]-=fee;
19         totalSupply+=share-fee;
20         earning+=fee;
21     }
22     function withdraw(uint share) public {
23         ... //fee computation
24         balance[msg.sender]-=share;
25         balance[msg.sender]-=fee;
26         earning+=fee;
27         totalSupply-=share+fee;
28         return swap_T_4_T0(share);
29     }
30     function accounting() public {
31         uint dividend = calc_dividend(earning, totalSupply,
32             balance[msg.sender]);
33         earning-=dividend;
34         balance[msg.sender]+=dividend;
35         uint interest = calc_interest(debt[msg.sender]);
36         debt[msg.sender]+=interest;
37         earning+=interest;
38     }
39     function swap_T0_4_T(uint amount) public {
40         price=IERC20.balanceOf(T)/IERC20.balanceOf(T0); // T0*T=k
41         return amount * price;
42     }
43     function loan(uint share, uint collateral) public {
44         ... //fee computation
45         new_collateral=collateral[msg.sender]+collateral;
46         new_debt = debt[msg.sender]+ share + fee;
47         if (new_debt*100>=new_collateral*75) return;
48         collateral[msg.sender] = new_collateral;
49         debt[msg.sender] = new_debt;
50         totalSupply-= share;
51         totalDebt + = share + fee;
52         earning+=fee;
53     }
54     function payoff (uint share) public {
55         ... //fee computation
56         debt[msg.sender]- = share-fee;
57         totalDebt - =share-fee;
58         totalSupply + = share-fee;
59         earning+=fee;
60     }
61     function liquidate (address account) require owner public {
62         if (debt[account]*100<collateral*75) return;
63         totalSupply+= debt[account];
64         earning+=collateral[account]-debt[account];
65         totalDebt-= debt[account];
66         debt[account] = 0; collateral[account] = 0;
67     }
68 }

```

Figure 3: Bank-like Conceptual Model for DeFi Contracts

in Figure 3. It is worthy noting that the code is not intended to be complete or sound, but rather provides a vehicle for the later discussions of financial properties.

State Variables. The global variables in lines 2 - 12 denote a number of key book-keepings that a bank has to maintain. Later in this section, we will show that they become the extended types in our system. In particular, T on line 2 is the base token of the bank, denoting the currency of bank, similar to the base currency in the Pools contract in Figure 1, while $T0$ on line 3 represents a universally utilized currency, analogous to USDC or ETH in smart contracts and USD in real life. Variable fee_rate is a ratio used in fee calculation. Usually, fee charged to the customer that initiates a transaction is calculated as a percentage of the transaction. $Interest_rate$ is the ratio used to calculate interest of debt. Line 6 defines the bank's earnings, which is typically incremented through fee and interest accumulations. Variable $totalSupply$ denotes the total amount of asset in the contract-specific token T . Similarly, $totalDebt$ on line 8 represents the total amount of debt in T token that users have accrued. On line 9, $price$ represents a conversion rate between $T0$ and T . On lines 10-12, we define a set of mappings that are used to store information specific to each user, including balance, debt, and collateral.

Deposit. Lines 14 - 21 denote how a customer deposits asset, which may be loaned to others to harvest interest. Note that $msg.sender$ is a standard Solidity term that denotes the user, namely, the sender of the transaction message (i.e., function call). It is usually called with a deposit amount in a universal currency (i.e., amount in $T0$), analogous to the user depositing US dollars. The amount is first converted to a share of the overall asset in the bank on line 15, in the bank's currency T . Intuitively, it denotes what portion of the bank's overall asset is owned by the user. On line 16, fee_ratio and then fee are updated (typically based on the utilization of asset). Then the user balance, total asset, and earnings are updated. Note that the fee is taken from the user and saved to the bank. As we will show later, many operations in smart contracts share a similar nature to the deposit function, although their implementations may be orders of magnitude more complex. Hence, if we can type variables in smart contracts to balance, fee, total supply etc., like the variables in our function. We can check properties such as *fee should be taken from balance*, an invariant across implementations.

Withdraw. Lines 22 - 29 define how a user withdraws assets. It is almost symmetric to `deposit()`. Note that the fee is taken from the user's balance.

Accounting. Lines 30 - 37 denote the internal regular accounting of the bank, determining how balances earn dividends and debts increase with accrued interest. In particular, dividend calculated on line 31 denotes the distribution of bank's earnings to the user (lines 32 and 33). Interest calculated on line 34 increases the user's debt on line 35 and the bank's earnings. Note that the functions `calc_dividend()` and `calc_interest()` (including function `update_fee()` on line 16) are not defined as they denote bank-specific protocols. When we instantiate the conceptual model to various smart contracts, different contracts have their own innovative and project-specific definitions of those functions.

Swap. Lines 38 - 41 show how an amount of the universal coin $T0$ is converted to the base token T . First, the price of T in terms of $T0$ must be computed. In finance, a rule is typically followed to determine the price of an asset x when trading it with another asset y . That is, the product of total amounts of x and y on market

remains unchanged by the trade [53]. To ensure this invariant, the price of trading x for y is the total amount of y divided by the total amount of x . Intuitively, if x is traded for y , x amount decreases and y amount increases, x shall become more expensive, and vice versa. Hence, the price of token T in terms of $T0$ is total T divides total $T0$, demonstrated on line 39.

Loan. Lines 42 - 52 show how a user initiates a loan provided an amount of collateral. The loan amount share (in T) and the collateral amount collateral are given on line 42. Line 46 checks if the total debt is currently lower than 75% of the total collateral. If not, the user is not allowed to borrow. If the loan is granted, the user's collateral and debt are updated on lines 47-48. The $totalSupply$ of the T token is decreased by the loaned amount on line 49, and the total debt $totalDebt$ is increased by the loaned amount and the fee on line 50. Finally, the earnings are incremented on line 51.

Payoff. Paying off debt is largely symmetric to taking a loan, except that it does not need to check the health of account. Note that the fee is added to the debt (the same as in `loan()`).

Liquidate. Lines 60 - 66 show how an account's collateral can be liquidated if the debt is not paid off in time. The `require owner` modifier on line 61 means that only the bank owner can perform this operation. During liquidation, the health of the account is first checked on line 61 to decide if it should be liquidated. If so, the collateral is split to two parts, the first part paying off the debt and going to the total supply, and the second part goes to the bank's earnings. Then the debts and collateral are reset, as referenced on lines 64 and 65, respectively.

3.2 Instantiation of the Conceptual Model to DeFi Contracts

In the following section, we reason how our model can be instantiated to various kinds of smart contracts. According to Defillama [3], a DeFi analytics platform, there are 13 different types of smart contracts. We take the 5 most popular project types (by [7]), namely, *yield and yield aggregators*, *lending*, *dexes*, *services*, and *derivatives*, and discuss how our model can be used to model their basic functionalities. The contract categories and their typical operations are listed in Table 1.

Yield and Yield Aggregators. These projects allow users to stake funds into smart contracts, and then use the aggregated funds to generate yield as profit. They rely on *strategies*, which are automated investment strategies to earn rewards, incentives, or interest in other smart contracts. One example of a yield strategy would be contributing funds to a liquidity pool in another smart contract and receiving fees from transactions involving that pool. Once the yield strategy ends, the yield is then made available to withdraw, with any unclaimed yield being used to reinvest in the user's account. Some examples of yield and yield aggregators include *Convex Finance* (TVL¹ \$3.7b) and *Sushi BentoBox* (TVL \$76.2m).

Typical functions within these projects include: *stake*, *withdraw*, *invest*, and *reinvest*. Observe that there are almost direct mappings of these operations to those in the bank-like model. For example, *stake* corresponds to `deposit()`, and *invest* and *reinvest* are just special forms of withdraws as they entail reducing the user's balance

¹TVL, or Total Value Locked, represents the amount of capital in a smart contract.

and sending the reduced amount out (e.g., to a strategy account). As such, we can check behavior correctness by making sure the bookkeepings in those functions follow a similar fashion to those inside `deposit()` and `withdraw()` in Figure 3. The Vader project in section 2 is such an example. Function `addLiquidity()` in Figure 1 corresponds to `deposit()` in Figure 3. Observe that the base and token in the former correspond to `T0` in the latter (as they are the tokens the user deposits) and the `liquidity` token corresponds to `T`. The invocation to `calcLiquidity()` on line 6 in Figure 1 corresponds to the `swap` function call on line 15 in Figure 3. As such, we can check balances are correctly updated and fee is properly charged. We want to point out that the above discussion is conceptual, and `ScType` does not require explicitly constructing such correspondences. Instead, they are implicitly encoded by our extended types. More will be discussed in our type system section.

Dexes. These projects host an exchange market between two currencies. Dexes support two types of users: liquidity *suppliers* and *traders*. Suppliers deposit amounts of both token types in order to grow the contract’s supply pool, receiving a portion of the trading fees as income. Traders deposit one type of token into the pool, and receive the corresponding amount of the other token, minus some fee. Examples of deployed dex projects include Uniswap (TVL \$3.83b) and Balancer (TVL \$1.023b). Typical functions in Dexes include *swap*, *addLiquidity*, and *removeLiquidity*. The last two correspond to `deposit()` and `withdraw()` of our model, and *swap* is equivalent to depositing in one token, swapping to another token using a function similar to `swap_T0_4_T()` in our model, and then withdrawing in the later token.

Lending. Lending projects allow users to loan money, placing a collateral at stake. These projects are directly comparable to banks. Some of the most well known lending projects include AAVE (TVL \$5.867b) and Compound Finance (TVL \$2.282b). Lending projects support functions such as *lend*, *liquidate*, and *repay*.

Services. These projects facilitate some services, such as a wallet and a game. They regulate the backend flow of funds of their corresponding applications. Popular service projects include Instadapp (TVL \$2.082b) and DefiSaver (TVL \$103m). Typical functions of service projects include: *deposit*, *withdraw*, *buy*, and *sell*. They can be expressed with operations in our bank model.

Derivatives. Derivative contracts (e.g., futures and options) allow users to speculate on price movement, interest rates, and other financial variables without directly owning the underlying asset. Some popular derivative projects include GMX (TVL \$583.22m) and dYdX (TVL \$357.21m). Their typical functions include: *trade*, *bid*, *liquidate*, *withdraw*, and *deposit*. A trade function, like `applyTrade()` in Figure 2, can be modeled by depositing one token, swapping to another token, and then withdrawing the later token. As such, checking the property of fee in this procedure, namely, *fee should always be at the cost of user*, regardless buy, sell, deposit, or withdraw, identifies the bug in Figure 2.

3.3 Token Unit and Scaling Factor

Besides the financial meanings, our type system also captures the implicit token units and scaling factors for individual variables.

```
1 usdcAmount = USDC.balance();
2 scaledUSDCAmount = USDC.balance() * 10^12;
3 totalAmount = usdcAmount + scaledUSDCAmount;
```

Figure 4: Bank Like DeFi Model

<Program>	$P ::= S$
<Statement>	$S ::= S_1; S_2 \mid x := v \mid x := y \mid x := a \mid$ $x := y \text{ op } z \mid x := \text{IERC20}(y_1).\text{balanceOf}(y_2) \mid$ $\text{IERC20}(y_1).\text{transfer}(y_2, x) \mid$ $\text{if}(x) S1; \text{else } S2; \mid \text{while}(x) S;$
<Var>	x, y, z
<Value>	$v \in \{1, 5, 10^{18}, \dots\}$
<Address>	a
<Comparison>	$\bowtie \in \{>, <, ==, !=, \dots\}$
<Binop>	$op \in \{+, \times, \bowtie, \dots\}$

Figure 5: Language

Token Units. On the blockchain, the majority of currencies used are tokens. Tokens have monetary value, and can be exchanged for real world currencies such as USD. Tokens most commonly used include USDC (USD Coin) and ETH (Ethereum). DeFi projects utilize these tokens, or implement their own tokens in order to supply liquidity and fuel their economies. In Solidity, these tokens are handled as integers, despite potentially being of different currencies. This is because as of current, programming languages lack built-in models for handling (monetary) units. For example, amounts of USDC and ETH are both represented using the same primitive integer type in Solidity. This may potentially allow vulnerabilities to arise when such amounts are incorrectly handled. Just like how meters and feet cannot be directly compared, even though they are both measurements of length, directly comparing two amounts of different token units should not be allowed either. To prevent such problems, each variable has its token unit in our type system.

Scaling Factor. In Solidity, using a *scaling factor* is a common technique to handle decimal numbers without using floating-point arithmetic, which is not supported in the language. A scaling factor is a power of 10 that is multiplied to an actual token amount. Many tokens have a default scaling factor; the USDC token has a factor of 10^6 . This means 1 USDC (equivalent to 1 USD in real life) is internally denoted by a value of 10^6 . The WETH token has a scaling factor of 10^{18} . Hence, in order to compute the price between USDC and WETH, USDC needs to be scaled by a factor of 10^{12} for the scaling factors to be equal. However, similar to token units, the onus of using scaling factors properly is completely on developers, who likely make mistakes such as forgetting to multiply/divide by the right scaling factor. We demonstrate this in Figure 4. An amount of USDC, `usdcAmount` is computed on line 1. Recall that the default scaling factor of USDC is 10^6 . On line 2, a variable `scaledUSDCAmount` is set to the USDC amount scaled by a factor of 10^{12} , making the factor of `scaledUSDCAmount` 10^{18} . Finally, these two amounts are added on line 3. This should not be allowed, since the two tokens have different scaling factors. However, Solidity compiles and runs the code without issue. Therefore we model scaling factors explicitly in our type system to prevent such errors.

3.4 Type System

Definitions. Figure 5 presents a language to facilitate discussion. Although `ScType` supports the complex syntax of Solidity, we use a simplified language for discussion. In particular, we represent

$\langle \text{Extended Type} \rangle$	$\tau ::= \langle f, s, u, a \rangle$
$\langle \text{Financial Type} \rangle$	$f \in \{\text{RawBal}, \text{AccBal}, \text{NetBal}, \text{T-Supply}, \text{Fee}, \text{Debt}, \text{Interest}, \text{Dividend}, \text{Price}, -, \dots\}$
$\langle \text{Scaling factor} \rangle$	$s \in \mathbb{Z} \cup \{-\}$
$\langle \text{Token Unit} \rangle$	$u := u \times u \mid u \div u \mid t_a \mid t_{this} \mid -$

Figure 6: Extended Types

variables of all types as *Var*. Only variables with primitive types of integers and addresses will be typed by *ScType*, since other primitive types such as boolean and string do not provide meaningful information for our purpose. We also define a selected set of statements within *Statement*. These statements are where most of the type checking and inference take place. The first three statements are assignments: $x := v$, $x := a$, and $x := y$ representing *Value*, *Address*, and *Variable* assignments, respectively. Following are binary operation statement $x := y \text{ op } z$, and two primitive function calls, namely, $x := \text{IERC20}(y_1).\text{balanceOf}(y_2)$ and $\text{IERC20}(y_1).\text{transfer}(y_2, x)$. The first one type-casts y_1 to an ERC20 token, intuitively some kind of currency, and then retrieves the balance of an account denoted by y_2 , which contains an address. The second one transfers x amount of y_1 token from the user that initiates the transaction (i.e., *msg.sender*) to an address denoted by y_2 . We model these functions as they directly disclose token units and financial types. Finally, we include statements representing conditionals and loops. Although our typing system handles other statements such as function calls, they are handled in a standard way, e.g., propagating types through parameters, and hence elided.

Extended Types. Figure 6 represents the types that we have developed for our system. In *ScType*, each variable is typed with a tuple of four: *Financial Type*, *Scaling Factor*, *Token Unit*, and *Address*. Financial type represents the monetary implication of the variable. Following the bank-like model in Figure 3, we list part of the supported financial types: raw balance (*RawBal*) denoting balance before accrual, balance after fee charged (*NetBal*), balance after dividend accrual (*AccBal*), total supply (*T-Supply*), transaction fee (*Fee*), debt, interest, and exchange price. The symbol ‘-’ denotes *not-applicable*, meaning the variable has no monetary implication. We elide some supported types such as collateral for discussion simplicity. Scaling factor represents the exponent of 10 that the variable is scaled by. Token unit represents the token unit of the variable. Token units are expressions of either an existing currency denoted by address a , represented by t_a , the currency of the current contract, represented by t_{this} , and their product or ratio, which are typically used in price computation. In contrast, sum and difference (of different token units) are not legitimate. The address aspect of the type denotes the address that a variable may hold.

Type System. Our type system is based on single-static-assignment representations by Slither [26] and flow-sensitive. In other words, different left-hand-side appearances of a variable are renamed and hence typed separately. In addition, it is field-sensitive and context-sensitive, although not directly reflected in our later discussion of type rules. It types individual public/external functions in a contract one-by-one. Typing one such function entails typing all the function directly/indirectly invoked as well. Some global variables and function parameters may not have their types automatically inferred, usually when such information is only implicitly assumed

$(R_1) \frac{}{x := a : \langle -, -, -, a \rangle}$	$(R_2) \frac{}{x := v : \langle -, \text{getScaleV}(v), -, - \rangle}$
$(R_3) \frac{y : \tau}{x := y : \tau}$	
$(R_4) \frac{y_1 : \tau_1 \quad y_2 : \tau_2 \quad \tau_1.u = \tau_2.u \quad \tau_1.s = \tau_2.s}{x := y_1 + y_2 : \langle \tau_1.f \oplus \tau_2.f, \tau_1.s, \tau_1.u, - \rangle}$	
$(R_5) \frac{y_1 : \tau_1 \quad y_2 : \tau_2 \quad \tau_1.u = \tau_2.u \quad \tau_1.s = \tau_2.s}{x := y_1 - y_2 : \langle \tau_1.f \ominus \tau_2.f, \tau_1.s, \tau_1.u, - \rangle}$	
$(R_6) \frac{y_1 : \tau_1 \quad y_2 : \tau_2}{x := y_1 \times y_2 : \langle \tau_1.f \otimes \tau_2.f, \tau_1.s + \tau_2.s, \tau_1.u \times \tau_2.u, - \rangle}$	
$(R_7) \frac{y_1 : \tau_1 \quad y_2 : \tau_2}{x := y_1 \div y_2 : \langle \tau_1.f \oslash \tau_2.f, \tau_1.s - \tau_2.s, \tau_1.u \div \tau_2.u, - \rangle}$	
$(R_8) \frac{y_1 : \langle -, -, -, a_1 \rangle \quad y_2 : \langle -, -, -, a_2 \rangle}{x := \text{IERC20}(y_1).\text{balanceOf}(y_2) : \langle \text{RawBal}, \text{getScaleA}(a_1), t_{a_1}, - \rangle}$	
$(R_9) \frac{y_1 : \tau_1 \quad y_2 : \tau_2 \quad \tau_1.u = \tau_2.u \quad \tau_1.s = \tau_2.s}{x := y_1 \triangleright y_2 : \langle -, -, -, - \rangle}$	

Figure 7: Type Rules

$\ominus$	RawBal	NetBal	AccBal	T-Supply	Fee	Debt	Dividend
RawBal	RawBal	$\times$	$\times$	T-Supply	$\times$	Debt	$\times$
NetBal	$\times$	NetBal	$\times$	T-Supply	$\times$	Debt	$\times$
AccBal	$\times$	$\times$	AccBal	T-Supply	$\times$	Debt	$\times$
T-Supply	$\times$	$\times$	$\times$	T-Supply	$\times$	$\times$	$\times$
Fee	NetBal	$\times$	AccBal	$\times$	Fee	$\times$	Dividend
Debt	RawBal	NetBal	AccBal	$\times$	$\times$	Debt	$\times$
Dividend	$\times$	$\times$	$\times$	T-Supply	Fee	Debt	Dividend

Table 2: Definition of Operator $\ominus$ (top_row $\ominus$ left_column)

$\text{getScaleV}(v)$	$= \begin{cases} n & v = 10^n \\ - & \text{otherwise} \end{cases}$
$\text{getScaleA}(a)$	$= \text{dictionary lookup of token } a\text{'s scaling factor}$

Figure 8: Helper functions used in type rules

as preconditions. In such cases, *ScType* prompts the user for their types. We call such user provided information *type annotations*, which are typical in static analysis and symbolic analysis. They can be extracted from documentation, comments, and even variable names. In section 4, we will show such manual efforts are limited. In the following discussion, we assume type annotations are in place for simplicity.

Figure 7 presents our rules for type inference and checking. Within our rules, $y : \tau$ represents variable y having an extended type τ . A statement $x := \dots : \tau$ means that τ is the resulting type of the statement that will be propagated to x . Special operations $\oplus$, $\ominus$, $\otimes$, and $\oslash$ represent the resulting financial meanings of addition, subtraction, multiplication, and division operations, respectively. We have included the definition of $\ominus$ in Table 2 as a reference. The definitions of remaining operators have been excluded for space, and can be found in our supplementary material [11].

Rule R_1 specifies that when assigning an address to a variable, the corresponding extended type τ tracks the address, which is later propagated to other places/variables through copy statements. Rule R_2 demonstrates that when assigning a value to a variable, τ records the scaling factor of the value, obtained through the helper function *getScaleV()* in Figure 8. Rule R_3 specifies that in a copy statement,

the left-hand-side variable inherits its type from the right-hand-side. Rules R4-R5 specify the rules for addition and subtraction. They are in a similar form. For example, rule R_5 specifies when a variable y_1 typed to τ_1 is subtracted by y_2 typed to τ_2 , their token units and scaling factors must be the same, and their financial types must be legitimate for subtraction as well (according to Table 2). In Table 2, the top row represents the minuend, the left column represents the subtrahend, and the intersection cell represents the result. A legal subtraction, such as `RawBal - Fee` yields `NetBal` (row 6 and column 2), which cannot be further subtracted by a fee (row 6 and column 3), indicated by the **X**. The intuition of the table can be derived from our DeFi model. For example, `Debt-Fee` is not allowed as fee shall increase debt, not decrease (see `loan()` in Figure 3). Some may wonder why we allow `RawBal-Debt` because paying off debt should be in the form of `Debt-RawBal`. The reason is that many DeFi projects allow users to over-pay their debts with the extra going to their balances. The extra is computed by `RawBal-Debt`. Rules R_8 and R_9 are for multiplication and division. For these two, we do not check consistencies of token units and scaling factors as multiplication and division of different tokens (with different scaling factors) are often necessary in computing trading/swapping price (see `swap_T0_4_T()` in Figure 3). The definitions of $\otimes$ and $\oslash$ are in our supplementary material. Specifically, we only allow multiplication/division within the same type of balance, not across. For example, `RawBal/RawBal` is allowed but `RawBal/NetBal` is not, as the ratio of the latter serves no purpose. Rule R_8 specifies the inference rule for `balanceOf(...)`. The conditions are that both y_1 and y_2 must be of the address type. The resulting variable x has the type $\langle \text{RawBal}, \text{getScaleA}(a_1), t_{a_1}, - \rangle$, representing that it is a raw balance, has a scaling factor that is looked up from a dictionary, which is easy to construct with one-time effort as there are only a few default scaling factors for popular tokens, and the token unit t_{a_1} . The rule for `transfer(...)` is similar and elided. Rule R_9 specifies that for comparison operations, the token units and scaling factors must be consistent. The resulting type is a null type, since *boolean* values do not have financial meaning. The rules for conditionals and loops are standard and elided.

Examples. Recall the Vader bug in Figure 1. Variables `addedBase` and `addedToken` on lines 4 and 5 are typed to $\langle \text{RawBal}, 0, t_{base}, - \rangle$ and $\langle \text{RawBal}, 0, t_{token}, - \rangle$, respectively, through the function calls of `getAddedAmount(...)`, which is a wrapper of `balanceOf(...)`. In other words, the two are raw balances with different token units. The addition on line 8 allows typing `totalBase` to token unit t_{base} . Similarly, `totalToken` has unit t_{token} and `totalLiquidity` has unit t_e , which is an expression of t_{base} and t_{token} . These 5 variables are the parameters passed to the `calcLiquidityUnits()` function. Through rule R_6 for multiplication, variables `part1`, `part2`, and `part3` all have type $\langle \text{RawBal}, 0, t_{token} \times t_{base}, - \rangle$. ScType reports an error during the 3-part calculation of unit on line 14. Specifically, the first part, $\text{TMP}_1 = P * \text{part1}$ is typed to $\langle \text{RawBal}, 0, t_e \times t_{token} \times t_{base}, - \rangle$ with no issue. The second part $\text{TMP}_2 = \text{TMP}_1 + \text{part2}$ is problematic by rule R_4 since there is a unit mismatch between TMP_1 and `part2`, with the unit $t_{token} \times t_{base}$.

For the example in Figure 2, `signedAmount` and `signedPrice` are typed to $\langle \text{RawBal}, 0, t_{base}, - \rangle$ and $\langle \text{Price}, 0, t_{quote} \div t_{base}, - \rangle$ from previous assignments not shown here. Similarly, `position.base`

and `position.quote` are typed to $\langle \text{RawBal}, 0, t_{base}, - \rangle$ and $\langle \text{RawBal}, 0, t_{quote}, - \rangle$; `quoteExchange` is typed to $\langle \text{RawBal}, 0, t_{quote}, - \rangle$ by rule R_6 . The code for function `getFee()` is not shown, but its return type (and hence the type of `fee`) is $\langle \text{Fee}, 0, t_{quote}, - \rangle$. The problematic statement is on line 13, within the true branch. The statement is split into two separate operations (by Slither). The first operation $\text{TMP}_1 = \text{position.quote} - \text{quoteChange}$ type-checks and results in $\langle \text{RawBal}, 0, t_{quote}, - \rangle$ by rule R_5 . The second operation $\text{newQuote} = \text{TMP}_1 + \text{fee}$ does not type-check as `Fee` cannot be added to `RawBal` (according to the rule located in the supplementary material [11]).

Limitations of Our Type System. Our system is based on the DeFi model in Figure 3, which only abstracts parts of the business models of DeFi projects. It is not quantitative such that our type system cannot detect pure calculation errors. However, our results show that more than half of accounting errors are type errors. In addition, there may be different designs even for the basic operations in Figure 3. For example, interest may not be directly added debt, but rather separately accounted. However, these design choices do not cause problems in our type rules. For instance, we allow interest to be added to debt (by a rule located in the supplementary material [11]) but we do not force such addition.

4 EVALUATION

We implement ScType in around 3,000 lines of Python code on Slither [26]. It consists of a type annotation parser, a type propagation system, and a type checking system. It is inter-procedural and cross-contract, meaning that it may automatically include functions from other contracts in analysis (if their code is available). It also handles arrays and object fields. To reduce the overhead of supporting context-sensitivity, it caches analysis results for each function. Details are elided. We aim to address the following research questions.

- **RQ1.** How effective is ScType in disclosing accounting bugs?
- **RQ2.** How efficient is ScType?
- **RQ3.** What are the categories and distributions of accounting bugs?
- **RQ4.** What is the capacity of our type system?
- **RQ5.** How effective is ScType in finding zero-days?

The system and the benchmarks are provided as supplementary material and will be released upon publication.

4.1 Experimental Setup

Benchmark. In the controlled experiments, we utilize the smart contract vulnerabilities collected by Zhang et al. in [78], which details 513 real-world bugs from 113 projects. Of which, 72 were categorized as accounting bugs. We preclude 15 of them due to the inability to be loaded by Slither or missing code. The detailed list of remaining bugs is in Table 3, following their chronological order of being reported. While a project may have multiple contracts, we run ScType on those in which accounting bugs were reported.

Baselines. We ran a few state-of-the-art static analyses on the set of bugs, such as Smartian [24], Slither [26], Oyente [51], and Mythril [4]. However, they could not find these accounting bugs. It is expected as these tools are built for other types of bugs. The results are also consistent with what was reported in [78].

Project Name	Summary	A+	Func checked	TW	FP	TP	NTE	MTE	Analysis Time
MarginSwap	Dex project for margin trading on Uniswap and Sushiswap	6	20	1	0	1	1	0	8.93s
Vader Protocol	Yield project for a collateralized stablecoin	17	58	4	1	2	2	0	28.95s
PoolTogether	Gaming service on yield interest	15	8	1	0	1	1	0	8.94s
Tracer	Derivative project that supports perpetual markets	10	32	1	0	1	1	1	110.57s
Yield Micro	Lending project supporting borrowing, lending, and liquidity	4	18	2	0	1	1	1	10.39s
Sushi Trident	Dex project for deploying personalized liquidity markets	24	196	0	0	0	2	0	19.64s
yAxis	Yield project where users' aggregated funds are used in strategies for yield	9	59	4	1	2	1	1	16.81s
Badger Dao	Yield project	9	22	2	0	1	0	0	14.61s
Wild Credit	Lending project relying on pairs of assets instead of a pool	17	960	4	2	1	0	0	45.37s
PoolTogether v4	Gaming service on yield interest	1	23	0	0	0	1	0	14.99s
Sushi Trident p2	Dex project for deploying personalized liquidity markets	12	51	10	4	2	2	0	19.68s
Swivel	Yield project that allows users to set orders for Yield claiming	5	78	2	0	1	0	0	15.03s
Covalent	Users delegate commissions to a Validators, which stakes the funds for interest	6	59	1	1	0	0	1	15.87s
Badger Dao p2	Stablecoin to keep the internal contract asset price from fluctuating	6	14	2	0	1	0	0	10.31s
Vader Protocol	Yield project for a collateralized stablecoin (Swap)	54	169	13	2	6	2	0	66.86s
yAxis p2	Yield project where users' aggregated funds are used in strategies for yield	2	1	0	0	0	1	0	9.58s
Malt Finance	Stablecoin for the contract token, Malt	7	63	0	0	0	2	0	11.998s
Perennial	Derivative project supporting synthetic token perpetual markets	3	1	1	0	1	0	0	9.35s
Sublime	Lending contract dependent on trust minimization	15	57	5	2	2	0	0	24.54s
Yeti Finance	Lending project made against a contract specific token	9	34	0	0	0	1	0	12.35s
Vader Protocol p3	Yield project for a collateralized stablecoin	5	37	4	0	3	1	0	16.38s
InsureDao	Insurance markets where buyers pay premium for protection against losses	11	72	0	0	0	1	0	14.86s
Rocket Joe	Dex project where users exchange funds in return for new project liquidity	7	29	3	1	1	0	0	2.860s
Concur Finance	Yield project	4	41	0	0	0	1	0	2.980s
Biconomy Hyphen	Cross Chain project where users can deposit and withdraw for pools on different chains	6	60	1	0	1	0	0	14.64s
Sublime	Lending project allowing users to create custom lending pools	8	91	0	0	0	1	0	25.84s
Volt	Dex project which conserves the value of user funds against inflation	1	40	0	0	0	1	0	18.86s
Badger Dao p3	Yield project	8	68	0	0	0	1	0	16.22s
Tigris Trade	Dex project utilizing off-chain oracles to provide real-time prices	11	86	2	0	1	1	0	7.024s
Total				63	14	29	24	4	

* Annotations (A+), Total Warnings (TW), False Positive (FP), True Positive (TP), Not Type Error (NTE), Missed Type Error (MTE)

Table 3: Evaluation Results

Initial Type Annotations. Although ScTYPE can automatically infer certain type information such as some token units and scaling factors, it may need the user to provide initial information such as financial types for some global variables and some function parameters (if they cannot be inferred). ScTYPE prompts the user for such information when it is missing and cannot be resolved by type inference. Users' efforts are one-time and recorded in a type file for reuse. The information is clear from project description and code comments in most cases, and hence the required user efforts are limited, as demonstrated by the number of annotations in Table 3 (in comparison to the number of functions type-checked). More automation is certainly feasible. For example, financial types can be inferred from variable names in many cases or using mining techniques such as [46, 48]. We leave it to our future work.

The experiments are conducted on a machine with AMD Ryzen 3975x and 512GB RAM.

4.2 RQ1: Effectiveness

The results from running ScTYPE on our benchmark are shown in Table 3. The projects are listed in the leftmost column, followed by a short summary of each project. The number of annotations made is shown in the column starting with A+. The number of functions checked is listed under the Func checked column, which includes functions that are called within other functions. The TW column indicates the number of type warnings by ScTYPE. Singular true positives (TP) and false positives (FP) may both generate multiple warnings (due to the cascading effect of a type error), hence the discrepancy between the warnings amount and the sum of true and

```

1  int usdvReserve; //an amount of USDV
2  int vaderReserve; //an amount of VADER
3  function getRewardShare(address token) external {
4      uint baseAmount = getBaseAmount(token);
5      if (isVader(token)) {
6          uint _share = calcShare(_baseAmount,
7                                  ..., vaderReserve);
8          ...
9      } else if (isUSDV(token)) {
10         uint _share = calcShare(_baseAmount, ..., usdvReserve)
11         ...
12     }
13     ...
14 }

```

Figure 9: False Positive Example in Vader Protocol p1

false positive amounts. The true positives are listed under the TP column. Accounting bugs that are not type errors (and hence out of scope for ScTYPE) are listed under the NTE, or Not-Type-Error column. Type bugs that are not able to be found with the current system are listed under the MTE, or Missed-Type-Error column.

Observations. In total, we run our tool on 29 projects, covering 57 accounting bugs. ScTYPE reports 29 TPs and 14 FPs. The FPs are mainly due to the path insensitive nature of the tool. We will illustrate with a case later. Even though ScTYPE cannot detect 28 of the 57 bugs, our inspection shows that 24 out of those 28 bugs are not type errors, belonging to other error categories such as pure math errors. Therefore, ScTYPE is able to successfully detect $29/(29+4)=87.9\%$ of accounting type errors. We argue that these results demonstrate the promise of ScTYPE as an attempt to addressing accounting bugs.

False positives. We manually inspect the false positives. We find that the lack of path sensitivity is a major reason. Figure 9 shows an example where ScTYPE fails to type-check variable `baseAmount` within the function `getRewardShare()`, which takes as parameter an address `token` that represents the address of an arbitrary currency. Variable `baseAmount` is calculated as an amount of the token currency on line 5 via the function `getBaseAmount()`. Then, `token` is compared to the addresses of two currencies, `VADER` and `USDV`, by the function calls to `isVADER()` and `isUSDV()` on lines 5 and 9, respectively. If the `if` branch is taken on line 5, the function `getRewardShare()` swaps `baseAmount` of the `VADER` token. If not, `USDV` is swapped. ScTYPE cannot resolve `baseAmount` to a unique token unit. Solving this problem may require path-sensitive analysis such as symbolic execution. We will leave this to our future work.

4.3 RQ2: Efficiency

To answer the research question regarding efficiency, we measure the cost of our tool, which is two-fold: the time of analysis and the total number of annotations. The former is shown in the last column of Table 3. Observe that the tool is very time affordable, with the maximum analysis time being less than 20 seconds. We point out that ScTYPE automatically type-checks all functions that are being called, regardless of whether or not they reside in the same contract. The number of annotations is also reasonable given the large number of functions checked. Most of the annotations can be derived with minimum one-time manual efforts.

```

1  function belowMaintenanceThreshold(uint256 loan, uint256
    collateral) external {
2      ...
3      - return 100 *collateral >= liquidationThresholdPercent *loan;
4      + return 100 *collateral < liquidationThresholdPercent *loan;
5  }

```

Figure 10: Not-Type-Error Example in MarginSwap

```

1  uint256 totalReserve;
2  uint256 strategyReserve;
3  function setCap(uint256 cap) external {
4      ...
5      diff = strategyReserve - cap
6      - totalReserve -= strategyReserve;
7      + totalReserve -= diff;
8      ...
9  }

```

Figure 11: Missed-Type-Error Example in yAxis p1

4.4 RQ3: Distribution of Type Errors

We categorize the 33 different accounting type bugs (29 found by ScTYPE and 4 missed) into 3 categories: token unit bugs, scaling factor bugs, and financial type bugs. Their numbers are 10, 12, and 11, respectively, which represent an approximately even distribution. The distribution strongly supports our current design. In addition, we find it a bit counter intuitive that token unit bugs are almost as common as the other two kinds, although they are simpler.

4.5 RQ4: Capacity of Type System

As shown in Table 3, ScTYPE cannot detect 28 accounting bugs. Out of the 28, 24 are not type errors and deemed out-of-scope, while 4 are type errors that currently cannot be handled. In this section, we provide two case studies to illustrate these two types of bugs. Figure 10 shows a typical not-type-error (NTE). The bug lies in that the developers used the wrong comparison. Other NTE types include coefficient errors and even use of wrong formulas. These bugs need stronger oracles than type rules, such as input-output pairs and formal specification of business models. Figure 11 shows a missed-type-error (MTE). This function computes the amount that a certain reserve of some *strategy* (e.g., a contract that yields) exceeds a cap and removes the excess from the total reserve. In particular, the specific reserve is denoted by variable `strategyReserve`, the cap is `cap`, the excess is `diff` and the total reserve is `totalReserve`. The bug lies in that developers incorrectly subtract `strategyReserve`, while they should subtract `diff`. Although it is beyond our current system, a stronger type system that models balance delta such as `diff` as well as balance upper-bound like `cap` may prevent this bug. We leave this to our future work.

4.6 RQ5: Finding Zero-days

To study the real-world impact of ScTYPE, we use it to audit a large real-world contract through Code4Rena. The project has over 10 thousands lines of code. We applied the technique on 9 contracts, found and reported 6 zero-days, with 4 of them leading to direct fund loss. Three of them are financial type errors and the other three are token unit bugs. We have created exploit inputs for these bugs as

proof-of-concepts. According to Code4Rena’s policy, details should not be made public until the judges inspect all the bug reports and the developers are given the chances to fix the bugs.

5 THREATS TO VALIDITY

There is *internal* threat to validity due to human mistakes in type annotations. In practice, these annotations are mostly obvious from documentation and variable names. For example, fee and debt variables tend to have subwords “fee” and “debt” in their variable names. In addition, there may be implementation errors. As wrong annotations/implementations lead to spurious type errors, the few false positives by ScTYPE indicate that the threat is mitigated. In the future, we plan to further reduce the human efforts (and hence the internal threat) by mining variables’ financial meanings. The *external* threat mainly lies in the subjects used in our study. We mainly use the bugs in [78], which may not be representative. The risk is mitigated as all projects in the benchmark are real-world applications, with many having high complexity. The bug reports had gone through multiple rounds of interactions between auditors, developers, and Code4rena judges. In addition, we recently apply ScTYPE to a very complex project and have encouraging results.

6 RELATED WORK

Detecting Business-related Vulnerabilities. Accounting bugs are related to business models. There have been pioneering efforts in detecting business related bugs. Wang et al. [69] proposed a fuzzing tool *Vultron*, which developed an interesting observation regarding *balance* and *transaction invariants*. In particular, the total balance of all the users and the contract should be the same, and transactions in or out of the contract should correspond to the same increase or decrease in total balance. This prevents bugs such as not updating a contract’s balance after a withdraw. They later developed mining techniques to infer these invariants [46–48]. Fairness bug detection [49] aimed to detect unfair behaviors for game-like contracts having multiple participants. Sun et al. [44] developed a method to detect smart contract vulnerabilities based on a swap invariant and a transfer invariant, or logical rules that must be followed in order to facilitate proper functions. The technique can detect overflows and unprotected asset increases. In comparison, ScTYPE can detect type problems that are largely complementary to the above works. Verification techniques [30, 32, 37, 38, 62, 64, 66, 72] are capable of detecting a wide spectrum of bugs including accounting bugs if the user can provide the specifications. In comparison, ScTYPE encodes properties in its type rules.

Abstract Type Inference and Checking. ScTYPE is essentially an abstract type system [16, 31, 33, 40–42, 50, 56, 59, 61] that derives abstract types with much richer semantics than primitive types. However, existing techniques do not focus on smart contracts, which have very unique finance oriented semantics. Tan et al. [64] developed a refinement type system known as *SolType* for Solidity. It models low level relationships between integers and checks for overflows/underflows.

Smart Contract Bug Finding. ScTYPE is related to smart contract bug finding in general, including static analysis [18, 23, 26, 27, 35, 39, 60, 65, 68, 71, 79], fuzzing [24, 28, 29, 34, 36, 45, 54, 67, 69, 70, 74, 75],

and symbolic execution [1, 17, 19, 43, 51, 52, 55, 61, 67, 73]. In contract, ScTYPE focuses finding accounting bugs, complementary to these techniques.

Bug Studies. We are inspired by a recent study on smart contract vulnerabilities [78], which showed the prevalence of accounting bugs and the difficulty of finding them, and also by a list of other comprehensive studies of various kinds of smart contract bugs and programming practices [20–22, 25, 57, 58, 76, 77].

7 CONCLUSION

We develop an abstract type inference and checking technique to detect accounting bugs in smart contracts, a kind of bug difficult for existing automatic tools. The technique models token units, scaling factors, and financial meanings of individual variables and checks type consistencies. Our results show that more 58% of known accounting bugs are type errors, and our tool detects 87.9% of these type errors. It also finds 6 zero-days.

REFERENCES

- [1] [n. d.]. Blockchain Technology Solutions. <https://consensys.net/>
- [2] [n. d.]. Code4rena. <https://code4rena.com>
- [3] [n. d.]. DefiLlama. <https://defillama.com/>
- [4] 2022. ConsenSys/mythril. <https://github.com/ConsenSys/mythril>
- [5] 2023. Biconomy Project. <https://github.com/ZhangZhaoSJTU/Web3Bugs/tree/main/contracts/97>
- [6] 2023. Coinmarketcap. <https://coinmarketcap.com/>
- [7] 2023. defillama/categories. <https://defillama.com/categories>
- [8] 2023. Funding Fee. <https://www.binance.com/en/blog/futures/what-are-funding-fees-in-binance-futures-6595842576313788144>
- [9] 2023. Perpetual Markets. <https://milkroad.com/funding/perpetual-contracts/>
- [10] 2023. Q2. <https://cointelegraph.com/news/crypto-hacks-and-exploits-snatch-over-300m-in-q2-2023-report>
- [11] 2023. Supplementary Material. https://github.com/NioTheFirst/ScType/blob/main/icse2024-paper1049-supplementary_material_upload.pdf
- [12] 2023. Tigris Trade Project. <https://github.com/ZhangZhaoSJTU/Web3Bugs/tree/main/contracts/192>
- [13] 2023. Tracer Project. <https://github.com/ZhangZhaoSJTU/Web3Bugs/tree/main/contracts/16>
- [14] 2023. uranium. <https://www.coindesk.com/markets/2021/04/28/binance-chain-defi-exchange-uranium-finance-loses-50m-in-exploit/>
- [15] 2023. Vader Protocol P1 Project. <https://github.com/ZhangZhaoSJTU/Web3Bugs/tree/main/contracts/5/vader-protocol>
- [16] Miltiadis Allamanis, Earl T. Barr, Soline Ducousso, and Zheng Gao. 2020. Typilus: neural type hints. *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (2020).
- [17] Priyanka Bose, Dipanjan Das, Yanju Chen, Yu Feng, Christopher Kruegel, and Giovanni Vigna. 2022. Sailfish: Vetting smart contract state-inconsistency bugs in seconds. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE.
- [18] Lexi Brent, Anton Jurisevic, Michael Kong, Eric Liu, Francois Gauthier, Vincent Gramoli, Ralph Holz, and Bernhard Scholz. 2018. Vandal: A scalable security analysis framework for smart contracts. *arXiv preprint arXiv:1809.03981* (2018).
- [19] Jialiang Chang, Bo Gao, Hao Xiao, Jun Sun, Yan Cai, and Zijiang Yang. 2019. sCompile: Critical Path Identification and Analysis for Smart Contracts. In *ICFEM (Lecture Notes in Computer Science, Vol. 11852)*. Springer, 286–304.
- [20] Jiachi Chen, Xin Xia, David Lo, and John C. Grundy. 2020. Why Do Smart Contracts Self-Destruct? Investigating the Selfdestruct Function on Ethereum. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31 (2020), 1 – 37.
- [21] Jiachi Chen, Xin Xia, David Lo, John C. Grundy, Xiapu Luo, and Ting Chen. 2022. Defining Smart Contract Defects on Ethereum. *IEEE Trans. Software Eng.* 48, 2 (2022), 327–345.
- [22] Jiachi Chen, Xin Xia, D. Lo, John C. Grundy, and Xiaohu Yang. 2021. Maintenance-related concerns for post-deployed Ethereum smart contract development: issues, techniques, and future challenges. *Empirical Software Engineering* 26 (2021).
- [23] Ting Chen, Xiaoqi Li, Xiapu Luo, and Xiaosong Zhang. 2017. Under-optimized smart contracts devour your money. In *SANER. IEEE Computer Society*, 442–446.
- [24] Jaeseung Choi, Doyeon Kim, Soomin Kim, Gustavo Grieco, Alex Groce, and Sang Kil Cha. 2021. SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In *ASE. IEEE*.

- [25] Wesley Dingman, Aviel Cohen, Nick Ferrara, Adam Lynch, Patrick Jasinski, Paul E. Black, and Lin Deng. 2019. Classification of Smart Contract Bugs Using the NIST Bugs Framework. In *SERA*. IEEE, 116–123.
- [26] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: a static analysis framework for smart contracts. In *WETSEB@ICSE*. IEEE / ACM.
- [27] Neville Grech, Michael Kong, Anton Jurisevic, Lexi Brent, Bernhard Scholz, and Yannis Smaragdakis. 2018. Madmax: Surviving out-of-gas conditions in ethereum smart contracts. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018).
- [28] Gustavo Grieco, Will Song, Artur Cygan, Josselin Feist, and Alex Groce. 2020. Echidna: effective, usable, and fast fuzzing for smart contracts. In *ISSTA*. ACM, 557–560.
- [29] Alex Groce and Gustavo Grieco. 2021. echidna-parade: a tool for diverse multicore smart contract fuzzing. In *ISSTA*. ACM, 658–661.
- [30] Shelly Grossman, Ittai Abraham, Guy Golan-Gueta, Yan Michalevsky, Noam Rinetzky, Mooly Sagiv, and Yoni Zohar. 2018. Online detection of effectively callback free objects with applications to smart contracts. *Proc. ACM Program. Lang.* 2, POPL (2018), 48:1–48:28.
- [31] Philip J. Guo, Jeff H. Perkins, Stephen McCamant, and Michael D. Ernst. 2006. Dynamic inference of abstract types. In *International Symposium on Software Testing and Analysis*.
- [32] Ákos Hajdu and Dejan Jovanovic. 2019. solc-verify: A Modular Verifier for Solidity Smart Contracts. In *VSTTE (Lecture Notes in Computer Science, Vol. 12031)*. Springer, 161–179.
- [33] Sudheendra Hangal and Monica S. Lam. 2009. Automatic dimension inference and checking for object-oriented programs. *2009 IEEE 31st International Conference on Software Engineering* (2009), 155–165.
- [34] Jingxuan He, Mislav Balunovic, Nodar Ambroladze, Petar Tsankov, and Martin T. Vechev. 2019. Learning to Fuzz from Symbolic Execution with Application to Smart Contracts. In *CCS*. ACM, 531–548.
- [35] Jing Huang, Kuo Zhou, Ao Xiong, and Dongmeng Li. 2022. Smart contract vulnerability detection model based on multi-task learning. *Sensors* (2022).
- [36] Bo Jiang, Ye Liu, and Wing Kwong Chan. 2018. Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In *ASE*. IEEE.
- [37] Jiao Jiao, Shuanglong Kan, Shang-Wei Lin, David Sanán, Yang Liu, and Jun Sun. 2020. Semantic Understanding of Smart Contracts: Executable Operational Semantics of Solidity. *2020 IEEE Symposium on Security and Privacy (SP)* (2020), 1695–1712.
- [38] Jiao Jiao, Shang-Wei Lin, and Jun Sun. 2020. A Generalized Formal Semantic Framework for Smart Contracts. *Fundamental Approaches to Software Engineering* 2020 (2020), 75–96.
- [39] Sukrit Kalra, Seep Goel, Mohan Dhawan, and Subodh Sharma. 2018. Zeus: analyzing safety of smart contracts.. In *NDSS*.
- [40] Michael Karr and David B. Loveman. 1978. Incorporation of units into programming languages. *Commun. ACM* 21 (1978), 385–391.
- [41] Sayali Kate, John-Paul Ore, Xiangyu Zhang, Sebastian Elbaum, and Zhaogui Xu. 2018. Phys: Probabilistic Physical Unit Assignment and Inconsistency Detection. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Lake Buena Vista, FL, USA) (*ESEC/FSE 2018*). 563–573. <https://doi.org/10.1145/3236024.3236035>
- [42] Andrew J. Kennedy. 1994. Dimension Types. In *European Symposium on Programming*.
- [43] Johannes Krupp and Christian Rossow. 2018. teEther: Gnawing at Ethereum to Automatically Exploit Smart Contracts. In *USENIX Security Symposium*. USENIX Association, 1317–1333.
- [44] Jin lei Sun, Song Huang, Xingya Wang, Meijuan Wang, and Jinhu Du. 2022. A Detection Method for Scarcity Defect of Blockchain Digital Asset based on Invariant Analysis. *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)* (2022), 73–84.
- [45] Chao Liu, Han Liu, Zhao Cao, Zhong Chen, Bangdao Chen, and Bill Roscoe. 2018. Reguard: finding reentrancy bugs in smart contracts. In *ICSE-Companion*. IEEE.
- [46] Ye Liu. 2022. A Unified Specification Mining Framework for Smart Contracts. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (2022).
- [47] Ye Liu and Yi Li. 2022. InvCon: A Dynamic Invariant Detector for Ethereum Smart Contracts. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (2022).
- [48] Ye Liu, Yi Li, Shang-Wei Lin, and Cyrille Artho. 2022. Finding permission bugs in smart contracts with role mining. *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (2022).
- [49] Ye Liu, Yi Li, Shang-Wei Lin, and Rong-Rong Zhao. 2020. Towards automated verification of smart contract fairness. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2020).
- [50] Benjamin Livshits, Aditya V. Nori, Sriram K. Rajamani, and Anindya Banerjee. 2009. Merlin: specification inference for explicit information flow problems. In *ACM-SIGPLAN Symposium on Programming Language Design and Implementation*.
- [51] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*.
- [52] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. 2019. Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *ASE*. IEEE.
- [53] Richard F Muth. 1964. The derived demand curve for a productive factor and the industry supply curve. *Oxford Economic Papers* 16, 2 (1964).
- [54] Tai D. Nguyen, Long H. Pham, Jun Sun, Yun Lin, and Quang Tran Minh. 2020. sFuzz: an efficient adaptive fuzzer for solidity smart contracts. In *ICSE*. ACM, 778–788.
- [55] Ivica Nikolic, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. 2018. Finding The Greedy, Prodigal, and Suicidal Contracts at Scale. In *ACSAC*. ACM, 653–663.
- [56] John-Paul Ore, Carrick Detweiler, and Sebastian G. Elbaum. 2017. Lightweight detection of physical unit inconsistencies without program annotations. *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (2017).
- [57] Daniel Perez and Benjamin Livshits. 2021. Smart Contract Vulnerabilities: Vulnerable Does Not Imply Exploited.. In *USENIX Security Symposium*.
- [58] Valentina Piantadosi, Giovanni Rosa, Davide Placella, Simone Scalabrino, and Rocco Oliveto. 2022. Detecting functional and security-related issues in smart contracts: A systematic literature review. *Software: Practice and Experience* 53 (2022), 465–495.
- [59] Veselin Raychev, Martin T. Vechev, and Andreas Krause. 2015. Predicting Program Properties from "Big Code". *ACM SIGPLAN Notices* 50 (2015), 111–124.
- [60] Michael Rodler, Wenting Li, Ghassan O Karamé, and Lucas Davi. 2018. Sereum: Protecting existing smart contracts against re-entrancy attacks. *arXiv preprint arXiv:1812.05934* (2018).
- [61] Sunbeom So, Seongjoon Hong, and Hakjoo Oh. 2021. SmartTest: Effectively Hunting Vulnerable Transaction Sequences in Smart Contracts through Language Model-Guided Symbolic Execution. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association.
- [62] Sunbeom So, Myungho Lee, Jisu Park, Heejo Lee, and Hakjoo Oh. 2020. VERIS-MART: A Highly Precise Safety Verifier for Ethereum Smart Contracts. In *IEEE Symposium on Security and Privacy*. IEEE, 1678–1694.
- [63] Bryan Tan, Benjamin Mariano, Shuvendu Lahiri, Isil Dillig, and Yu Feng. 2021. SolType: Refinement Types for Solidity. *arXiv preprint arXiv:2110.00677* (2021).
- [64] Bryan Tan, Benjamin Mariano, Shuvendu K Lahiri, Isil Dillig, and Yu Feng. 2022. SolType: refinement types for arithmetic overflow in solidity. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 1–29.
- [65] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. 2018. SmartCheck: Static Analysis of Ethereum Smart Contracts. In *WETSEB@ICSE*. ACM, 9–16.
- [66] Palina Tolmach, Yi Li, Shang-Wei Lin, Yang Liu, and Zengxiang Li. 2020. A Survey of Smart Contract Formal Specification and Verification. *ACM Computing Surveys (CSUR)* 54 (2020), 1–38.
- [67] Christof Ferreira Torres, Antonio Ken Iannillo, Arthur Gervais, and Radu State. 2021. ConFuzzius: A Data Dependency-Aware Hybrid Fuzzer for Smart Contracts. In *EuroS&P*. IEEE, 103–119.
- [68] Petar Tsankov, Andrei Marian Dan, Dana Drachler-Cohen, Arthur Gervais, Florian Bünzli, and Martin T. Vechev. 2018. Securify: Practical Security Analysis of Smart Contracts. In *CCS*. ACM, 67–82.
- [69] Haijun Wang, Yi Li, Shang-Wei Lin, Lei Ma, and Yang Liu. 2019. Vultron: catching vulnerable smart contracts once and for all. In *ICSE-NIER*. IEEE.
- [70] Haijun Wang, Ye Liu, Yi Li, Shang-Wei Lin, Cyrille Artho, Lei Ma, and Yang Liu. 2020. Oracle-supported dynamic exploit generation for smart contracts. *IEEE Transactions on Dependable and Secure Computing* (2020).
- [71] Shuai Wang, Chengyu Zhang, and Zhendong Su. 2019. Detecting nondeterministic payment bugs in Ethereum smart contracts. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019).
- [72] Yuepeng Wang, Shuvendu K. Lahiri, Shuo Chen, Rong Pan, Isil Dillig, Cody Born, Imad Naseer, and Kostas Ferles. 2019. Formal Verification of Workflow Policies for Smart Contracts in Azure Blockchain. In *VSTTE (Lecture Notes in Computer Science, Vol. 12031)*. Springer, 87–106.
- [73] Zexu Wang, Bin Wen, Ziqiang Luo, and Shaojie Liu. 2021. MAR: A Dynamic Symbol Execution Detection Method for Smart Contract Reentry Vulnerability. In *International Conference on Blockchain and Trustworthy Systems*. Springer.
- [74] Valentin Wüstholtz and Maria Christakis. 2020. Harvey: a greybox fuzzer for smart contracts. In *ESEC/SIGSOFT FSE*. ACM, 1398–1409.
- [75] Yinxing Xue, Jiaming Ye, Wei Zhang, Jun Sun, Lei Ma, Haijun Wang, and Jianjun Zhao. 2022. xFuzz: Machine Learning Guided Cross-Contract Fuzzing. *IEEE Transactions on Dependable and Secure Computing* (2022).
- [76] Pengcheng Zhang, Feng Xiao, and Xiapu Luo. 2020. A Framework and DataSet for Bugs in Ethereum Smart Contracts. In *ICSME*. IEEE, 139–150.

- [77] Wuqi Zhang, Lili Wei, S. C. Cheung, Yepang Liu, Shuqing Li, Luanqi Liu, and Michael R. Lyu. 2022. Combatting Front-Running in Smart Contracts: Attack Mining, Benchmark Construction and Vulnerability Detector Evaluation. *IEEE Transactions on Software Engineering* 49 (2022), 3630–3646.
- [78] Zhuo Zhang, Brian Zhang, Wen Xu, and Zhiqiang Lin. 2023. Demystifying Exploitable Bugs in Smart Contracts. *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (2023), 615–627.
- [79] Ence Zhou, Song Hua, Bingfeng Pi, Jun Sun, Yashihide Nomura, Kazuhiro Yamashita, and Hidetoshi Kurihara. 2018. Security assurance for smart contract. In *NTMS*. IEEE.